

BuildMarshalAI

Presentation guide — speaking script and live-demo checklist

20 minutes · 5 speakers · about 4 minutes each · CSE Capstone, BUET · industry partner: BuildMarshalAI

How to use this guide. Each member has one section. Read **What to say** aloud. It is written to be spoken, at roughly 130–140 words a minute with the demo actions in between. **[Blue bracketed text]** is an action, not something to say. **Demo steps** is the exact click path. **If it fails** tells you what to do without losing time. Every page, button and answer here exists in the current build and was checked against it. **The whole demo was rehearsed end to end on 30 Sep 2026:** every step below passed on the real backend (GPU, model, local Whisper), with the Google steps on the simulated account. The step-by-step screenshots are in the companion *Demo Walkthrough* PDF.

Run sheet

Time	Member	Theme	On screen
0:00	Tonmoy (2105076)	Problem, idea, architecture overview	Architecture figure → homepage → sign in
4:00	Enayet (2105107)	Set up a project: people, roles, tasks, analytics	New Project → People → Add Task → Padma View Specialised Hospital Extension: Timeline, Statistics
8:00	Mukdho (2105064)	Ingestion and hybrid retrieval	Project Documents (upload, link) → Documents page → chat question → evidence viewer
12:00	Shafi (2105097)	Voice, chat actions, Google Workspace	Mic question → Google Workspace (Drive) → Meet from chat → Calendar
16:00	Ahtashamul (2105067)	Backend, generated document, conclusion	Generate Doc (Tender Summary) → User Roles → generated PDF → close
20:00	All	Questions	—

The story the demo tells

One continuous workflow, with no page visited twice for the same reason. A project manager creates **Bashundhara Clinic Annex** with a manager and team (Enayet). They bring its documents in and ask Marshal about the prepared **Padma View Specialised Hospital Extension** (Mukdho). They ask by voice and book a meeting from a sentence (Shafi). Finally they generate a cited Tender Summary (Ahtashamul). The architecture is introduced once (Tonmoy), and each speaker then points at the part of it their demo exercises.

Before the presentation — setup checklist

One person (suggested: Tonmoy) owns this list and ticks it off. A failed demo is almost always a skipped setup step.

The day before

- Rehearse the full 20 minutes at least once **on the demo laptop**, with a timer.
- Reconnect the Google account on the **Google Workspace** page. Google's testing-mode refresh tokens expire after 7 days (deviation D1 in our report), so do it again on the morning of the talk.
- Prepare the **presenter account**: a Head (Super Admin) user. Give it a name that is **not** in the pack's team list (e.g. *Demo Presenter*): if two users share a name, onboarding cannot tell which one is the manager or an assignee, and leaves them unset. Onboarding the pack adds its 10 team members, so the manager and assignee dropdowns have real choices.
- Prepare **Padma View Specialised Hospital Extension**, onboarded from demo/project-pack (Onboarding → upload the pack → Analyze → Review & complete → Confirm), so it has tasks, costs, a timeline and statistics. In rehearsal this took about 5½ minutes (8 documents, 141 records: 40 tasks, 40 task costs, 10 people, 8 procurement lines). Onboarding links its source documents to the new project automatically; check that **Project Documents** lists all 8.
- Make sure **06_Tender-Specification.pdf** is on the Documents page (so Mukdho can *link* it) and that **09_Site-Instruction_SI-014.docx** is at hand to *upload* live.
- Ask *What is the RFI response period on Padma View?* once and keep that conversation, as the backup for Mukdho.
- Generate one **Tender Summary** for Padma View Specialised Hospital Extension and keep the downloaded PDF in Downloads, as the backup for Ahtashamul.
- Book one rehearsal meeting from the chat, so the Calendar page has a backup event.
- Attach one small file in the chat (e.g. a site photo) and don't file it under a project, so Mukdho's **Unassigned** filter has something to show.
- Open the chat panel, start a **new chat** (the + icon) so the audience doesn't see rehearsal messages, then close the panel.
- Delete any earlier **Bashundhara Clinic Annex** so the live creation doesn't clash on the project code.

30 minutes before

- Run `START-BUILDMARSHAL.ps1 -StartBackend` from the package folder. It starts CLIProxyAPI, the frontend (port 5500), the local Whisper voice service (port 8903) and the backend (port 8000). Loading ColPali takes one to two minutes.
- Open `http://127.0.0.1:8000/api/health`. Check status: `healthy`, `database.connected: true` and `voice.available: true`.
- Open `http://localhost:5500` in Chrome, zoom to **110–125 %** so the back row can read it, and close every other tab. Sign out, so Tonmoy can sign in live.
- Allow the microphone for localhost (click the mic once and cancel), and test it in the room.
- Turn off notifications (Windows Focus Assist) and pause IDM or any other download manager.
- Have the architecture figure (the architecture page of this guide, or the report) open full-screen in a second window.

Backup environment (if the live backend is down)

- Run `python demo/demo_server.py --port 8901` (add `--reset` for fresh data) and open `http://127.0.0.1:8901`. It needs no GPU and no model.

- Sign in as `rafiqu1.islam@purbachalcon.example` with password `PadmaView2026!`. This is the seeded demonstration account; it only exists in the local demo database.
- It has the same pages and the same Padma View project. Chat answers are scripted from the pack, not generated, and Google and Microsoft are **simulated** (every response is marked so). Say so openly if you switch to it.
- Last resort: present from the report's screenshots, and describe each step while pointing at them.

The system architecture (show this during Section 1)

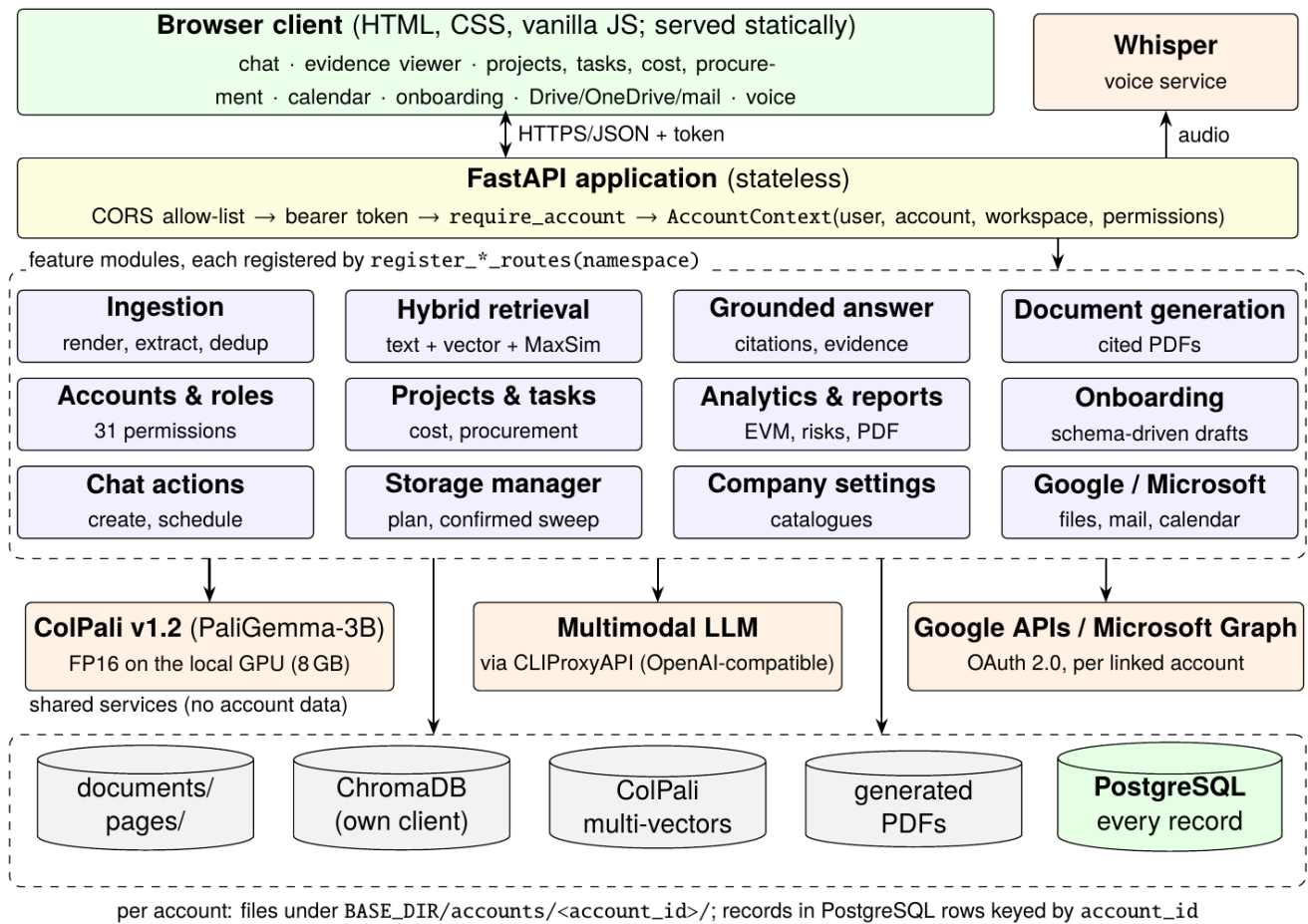


Figure 4.1 of the project report. Every request is resolved to one account's workspace before any module touches data; models are shared, data is not.

Architecture element	Explained by	Demonstrated when
Five tiers; account and permission layer; storage overview	Tonmoy	Sign-in resolves the account
Projects & tasks, project people, permissions, analytics	Enayet	New Project, People, Add Task, Statistics
Ingestion, ColPali, ChromaDB, hybrid retrieval, project scope, grounded answer	Mukdho	Upload / link, chat answer, evidence viewer
Browser client, Whisper voice service, OAuth integrations, confirmation protocol	Shafi	Mic question, Drive, Meet from chat
FastAPI app and module registration, isolation, PostgreSQL, generator abstraction, document generation	Ahtashamul	Generate Doc, User Roles, cited PDF

1

The problem, the idea, and the architecture at a glance

Md Mobasharul Islam Tonmoy (2105076) · Quality, documentation and communication · 0:00 – 4:00

Time budget: Problem and partner 0:45 · What we built 0:45 · Architecture walk-through 1:45 · Open the app, hand over 0:45

Script length: about 395 spoken words, roughly 2.9 minutes of speech; the rest is demo time.

What to show

- The architecture figure (the architecture page of this guide, or the report's Figure 4.1) on the projector.
- Then the live app: the homepage at <http://localhost:5500>, then **Open BuildMarshal**.

Architecture elements to explain

- **All five tiers, briefly:** browser client → stateless FastAPI API → account and permission layer (require_account → AccountContext) → feature modules → per-account storage.
- **The three model services beside them:** ColPali v1.2 on the local GPU, the multimodal LLM through CLIPProxyAPI, and the local Whisper voice service.
- **Storage:** every record lives in PostgreSQL; files, page images and the ChromaDB index live on disk under the account's own folder.

What to say

Good morning. Our project is BuildMarshalAI, built with our industry partner of the same name. I'm Tonmoy, and in the next twenty minutes the five of us will show you what we built, how it is put together, and then use it live on a real construction project.

Start with the problem. A construction project runs on documents: tender specifications, drawings, works programmes, cost plans, site instructions. A site engineer who needs one answer, say how many days the consultant has to reply to an RFI, has to find the right clause in hundreds of pages. And the tools the team uses to plan the project, for tasks, costs, people and meetings, know nothing about those documents.

So we built BuildMarshalAI: a project workspace with an assistant, Marshal, that answers from the project's own documents. Every answer comes with a citation and the exact page as evidence. It says so when the documents don't contain the answer. And nothing outward, like an email or a meeting, happens without your confirmation.

[Show the architecture figure.] Here is the whole system on one slide. At the top is the browser client, a single-page app in plain HTML, CSS and JavaScript. Every request goes to one FastAPI application. The first thing that application does is resolve your session token into an account context: your user, your account, your workspace, and your permissions. No module can touch data without going through it. That is how one installation serves several companies without their data ever mixing.

In the middle are the feature modules: ingestion, hybrid retrieval, grounded answers, document generation, projects and tasks, analytics, onboarding, and the Google and Microsoft integrations. Each one registers its own routes, so each can be built and tested on its own. Beside them sit three model services. ColPali reads document pages as images on the local GPU. A hosted multimodal

model writes the answers. And a local Whisper service turns speech into text. At the bottom is storage. Every record is a row in PostgreSQL, and each account's files and vector index live in that account's own folder.

My own part was quality. We have over seven hundred automated backend tests, plus browser suites that drive the real interface end to end. So what you are about to see is tested every time we change something.

[Switch to the browser. Show the homepage for five seconds, then click Open BuildMarshal and sign in.] This is the live system, running on our workstation with the GPU.

Demo steps

1. Before starting: the architecture figure is open in one window, and the browser at `http://localhost:5500` in another, already signed out.
2. At the architecture paragraph, point at the tiers top to bottom. Don't read the boxes out one by one.
3. Switch to the browser → homepage → **Open BuildMarshal** (the big button in the hero) → sign in with the prepared presenter account (a Head (Super Admin) user).
4. Land on **Projects** → **All Projects**. Leave it there for Enayet.

If it fails

- **Projector or laptop problem:** present the architecture from this printed guide; the demo laptop can be swapped in later.
- **Sign-in fails** (backend not ready): open `http://127.0.0.1:8000/api/health` in a tab. If it is not healthy, switch to the backup demo server (see the Setup page) and carry on there; the flow is the same.

Transition

"That's the shape of the system. Now let's use it the way a project manager would, starting with a brand-new project. Enayet?"

2

Setting up a project: people, roles, tasks, and the numbers

Md Enayet Alvee (2105107) · Project management and analytics · 4:00 – 8:00

Time budget: Create the project (manager, members) **1:15** · People tab **0:30** · Task with date validation **1:00** · Timeline + Statistics **1:15**

Script length: about 361 spoken words, roughly 2.7 minutes of speech; the rest is demo time.

What to show

- **All Projects → New Project:** create **Bashundhara Clinic Annex** live.
- Its **People** tab, then **Add Task** with the date checks.
- The prepared project **Padma View Specialised Hospital Extension:** its **Timeline** and **Statistics** tabs.

Architecture elements to explain

- **Feature modules:** projects, tasks, and project people (`tasks.py`, `project_management.py`, `project_people.py`).
- **Account layer:** every write is checked against the caller's permissions on the server. There are 31 permissions, and two built-in roles, Head (Super Admin) and Head (System Admin).
- **PostgreSQL:** projects, tasks and procurement are separate tables keyed by account. A multi-record change is one transaction.
- **Analytics module:** earned value, schedule health and rule-based risks (`project_analytics.py`).

What to say

Thanks, Tonmoy. I'm Enayet, and I built the project management and analytics side. Let's create a new project, a clinic annex for a client in Dhaka.

[All Projects → New Project. Type the name and code.] Notice the Project Manager field. You don't type a name here. It's a list of the account's real users. **[Open the dropdown, choose the manager.]** The project stores that user's ID, not a piece of text. So if someone is renamed, every project shows the new name, and nobody can be made manager of a project by a typo. Below it, I can add members straight away. **[Tick one or two members. Set the start and end dates. Save.]**

[Open the project → People tab.] The People tab shows everyone on the project, and why they're on it: the manager, the members we just added, and anyone who is later given a task. Only an explicit membership can be removed here. And someone who still has open work on the project can't be removed until those tasks are reassigned.

[Overview → Add Task.] Now a task. The assignee list only contains people on this project, so work can't be handed to a stranger. The date pickers are limited to the project's own dates. Watch what happens if I try to start before the project does. **[Set a start date one day before the project start and save. The warning appears.]** It's refused, and it tells me why. That rule is enforced on the server too. Even if someone bypasses the form and calls the API directly, a task outside its project's dates gets a 422 back. **[Fix the date and save. The task appears.]**

Every one of these actions is also checked against permissions on the server. We have 31 of them, down to which fields of a task a role may change and who may touch which costs. A Viewer can look at this project but not edit it, and the server enforces that, not just the buttons.

[Open Padma View Specialised Hospital Extension.] Here's a project that's further along. It was onboarded from its own documents, the tender, the programme and the cost plan, so it already has dozens of tasks with their costs. **[Timeline tab.]** The timeline draws every task against the project window. **[Statistics tab.]** And Statistics turns the records into earned value, planned against earned, budget and commitment, cost by trade, and an Attention list of rule-based risks, like overdue tasks or work running past the project's end. Every figure says what it's based on, so a manager can trust it.

Demo steps

1. **All Projects** → **New Project**. Name: **Bashundhara Clinic Annex**; code: **BCA-LIVE**; Project Manager: pick a prepared user; tick 1–2 members; Start **today**, End **three months from today**; Save.
2. Click **Bashundhara Clinic Annex** in the list → **People** tab. Point at the *Manager* and *Member* labels.
3. **Overview** → **Add Task**. Title **Site mobilisation**. Set the Start to **one day before** the project start → Save → the warning toast appears at the top centre.
4. Change the Start to the project's start date and the End to one week later; choose an assignee → Save.
5. Open **Padma View Specialised Hospital Extension** (All Projects) → **Timeline** tab (5 s) → **Statistics** tab. Scroll once to *Planned against earned value* and *Attention*.

If it fails

- **Project creation errors:** open **Projects** → **All Projects** and use the prepared project for everything; say "we created this one earlier".
- **Statistics slow to load:** press the tab's refresh button once. If it's still empty, describe the charts using the report's statistics screenshot.
- **Only one user available** in the dropdown: fine, make the presenter the manager and skip members.

Transition

"So the project is set up. But the real knowledge of a construction project sits in its documents. Mukdho will bring them in and ask Marshal about them."

3

Documents in, answers out: ingestion and hybrid retrieval

H.M. Aktaruzzaman Mukdho (2105064) · Retrieval and model lead · 8:00 – 12:00

Time budget: Upload + link existing **1:00** · Documents page **0:30** · Chat question + answer **1:15** · Evidence viewer + how retrieval works **1:15**

Script length: about 342 spoken words, roughly 2.5 minutes of speech; the rest is demo time.

What to show

- **Project Documents** on Bashundhara Clinic Annex: **Upload** a small file and **Link existing** documents.
- The **Documents** page: every document with all its projects, and the **Unassigned** filter.
- **Marshal Chat:** a question naming the project, the cited answer, then the **evidence viewer**.

Architecture elements to explain

- **Ingestion module** (`ingestion_formats.py` + notebook cell): every format is turned into page images plus text. Duplicates are recognised by a SHA-256 content hash and linked instead of stored twice.
- **ColPali v1.2 on the GPU:** one multi-vector per page. Pooled vectors go into the account's own ChromaDB, and the multi-vectors are cached on disk.
- **Hybrid retrieval** (`hybrid_retrieval.py`) has three channels: lexical scoring, pooled-vector candidates, and ColPali MaxSim re-ranking, fused into one score.
- **Project scope** (`chat_scope.py`): a question that names a project searches only the documents linked to it.
- **Grounded answer:** the top pages go to the multimodal LLM through CLIPProxyAPI, with numbered sources, so every citation maps to a real page.

What to say

Thanks, Enayet. I'm Mukdho, and I worked on how documents get in and how Marshal finds the right page. Our new project needs its documents.

[Open Bashundhara Clinic Annex → Project Documents → Upload, pick the site-instruction file.]

I can upload a file straight into the project. PDFs, Word, Excel, images and even audio all work. Every page is rendered to an image and its text is kept. Then ColPali, a vision-language retrieval model running on our GPU, embeds each page image. So scanned drawings and tables are searchable, not just clean text.

[Click Link existing, tick the tender specification, save.] Many documents are shared between projects, like a company's standard specification. So instead of uploading it again, I link the one that's already there. **[Open the Documents page.]** The Documents page shows every project each document belongs to, in full. The tender spec now says both Padma View Specialised Hospital Extension and Bashundhara Clinic Annex. It's still one document, with one set of vectors, just linked twice. And **[click Unassigned]** this filter shows the orphans, files nobody has filed under a project yet.

[Open Marshal Chat and type: What is the RFI response period on Padma View?] Now let's ask. I've named the project in the question. So Marshal searches only the documents linked to that project, and the answer says so at the top. **[Wait for the answer.]** Seven working days, and three for an urgent RFI on the critical path. The little numbers are citations. **[Click citation 1.]**

This is the evidence viewer. It opens the actual page the answer came from, highlights the matched terms, and shows why the page was selected. Retrieval has three channels. A lexical score catches exact clause words. A pooled vector search finds candidate pages quickly. Then ColPali compares the question against every patch of each candidate page, which is called MaxSim late interaction. We fuse the three. In our retrieval experiment, a page with the answer was in the top five for 95% of the questions, and the median answer time was about seven seconds. One honest finding: on clean, text-heavy pages, our lexical score alone was the strongest ranker at top-1. That's why the fusion leans on it when there's an exact match, and why we plan to make the weighting depend on how much text a page has.

Demo steps

1. **Bashundhara Clinic Annex** → **Overview** → **Project Documents** → **Upload** → choose 09_Site-Instruction_SI-014.docx from demo/project-pack (a small file that indexes quickly).
2. **Link existing** → search box: type *Tender* → tick **06_Tender-Specification.pdf** → Save.
3. Sidebar **Documents** → point at *Projects: ...* on the tender spec → click **Unassigned**, then **All**.
4. Open the chat panel → type exactly: *What is the RFI response period on Padma View?* → Send.
5. When the answer appears (≈ 7 s), point at *Searched only ...* and the citations → click **[1]** → the evidence viewer opens → point at the page, *Matched terms*, and *Why this page was selected*.
6. Close the viewer; leave the chat panel open for Shafi.

If it fails

- **Upload is slow** (GPU busy): skip it and say "this indexes in the background"; go straight to Link existing, which is instant.
- **Chat answer takes more than 20 s, or errors** (model proxy): open the chat **history** and reopen the conversation prepared before the talk, which has the same question already answered, then open the evidence viewer from it.
- **Wrong page cited**: say so honestly, then click the next citation. The scores are on screen, which makes the point about ranking.

Transition

"Typing works. But on a building site, your hands are usually full. Shafi will show how Marshal listens, and how it connects to the tools the team already uses."

4

The interface: voice, chat actions, and Google Workspace

Shafiul Tanvir (2105097) · Frontend and integrations · 12:00 – 16:00

Time budget: Voice question 1:15 · Google Workspace + Drive 1:00 · Meeting from chat, with confirmation 1:15 · Wrap-up of the interface 0:30

Script length: about 291 spoken words, roughly 2.2 minutes of speech; the rest is demo time.

What to show

- The **microphone** in Marshal Chat: speak a question, review the transcript, send it.
- **Google Workspace:** the connected account and the Drive folder browser (**Load files**, then open a folder).
- A meeting scheduled from a chat sentence, confirmed with a yes, then the **Calendar** page.

Architecture elements to explain

- **Browser client:** one single-page app with no build step. Every call goes through one API helper that attaches the session token.
- **Voice:** the browser records up to 60 s and posts it to the backend (/api/voice/transcribe), which forwards it to the **local Whisper service**, a separate process, so a voice failure can't take retrieval down.
- **Google / Microsoft modules:** OAuth 2.0 with PKCE. Tokens are encrypted at rest in PostgreSQL and never sent to the browser.
- **Confirmation protocol:** every outward action (email, event, meeting) is two-phase. The server returns a plan, and nothing happens until the user confirms.

What to say

Thanks, Mukdho. I'm Shafi, and I built the interface and the integrations. First, voice. [\[Click the microphone.\]](#) Notice the notification at the top: it's recording. [\[Say clearly: What are the working hours on site for Padma View?\] \[Click stop.\]](#)

The recording goes to our backend, and the backend passes it to a Whisper speech model running as its own service on this machine. [\[The transcript appears in the box.\]](#) And here's the important part. The transcript lands in the box for me to check. It is never sent on its own. On site, a misheard word shouldn't become a wrong question. [\[Press Send.\]](#) Eight in the morning to six in the evening, Saturday to Thursday, with Friday off, cited from the tender.

[\[Open Google Workspace in the sidebar.\]](#) Marshal also works inside the tools a team already uses. This account is connected to Google through OAuth. The access tokens are encrypted in our database, and the browser never sees them. [\[Click Load files, then a folder.\]](#) I can browse Drive folders here and import a file into a project, and it goes through exactly the same ingestion pipeline Mukdho just showed you.

[\[Back to the chat. Type: Schedule a Google Meet called RFI log review with \(teammate's email\) tomorrow at 10am.\]](#) Now a sentence that does something in the outside world. Marshal works out the title, the time and the attendee. And if something mandatory is missing, it asks, instead of guessing. [\[Marshal replies with the meeting it will create.\]](#) But it doesn't book anything yet. It shows me what it understood and waits for me to say yes. Every outward action in the system works this way, whether it's an email, an event, or a meeting. [\[Type yes and send.\]](#) Now it's booked, with a

real Meet link. [\[Open Calendar.\]](#) And the Calendar page shows it together with our project and task dates, in one view.

The same interface also creates records from sentences. For example, "create a task called pour slab on Padma View" opens a filled-in form, and only asks for what's missing. The idea throughout is that Marshal does the typing, but the person stays in charge.

Demo steps

1. Chat panel open → click the **mic** → wait for the *Recording...* toast → say: *What are the working hours on site for Padma View?* → click **stop**.
2. When the text appears in the box, pause for a second so people can read it → **Send**.
3. Sidebar **Google Workspace** → show the connected account → on the **Drive** tab press **Load files** (the table stays empty until you do) → click one folder name (don't import live, to save time).
4. Back in the chat → type: *Schedule a Google Meet called RFI log review with tomorrow at 10am* → answer any follow-up Marshal asks → when it shows the meeting it will create, reply **yes**.
5. Sidebar **Calendar** → point at the new meeting (and its Meet link) next to the project dates.

If it fails

- **Mic not picked up / room too noisy:** type the same question instead and say "the voice path is the same after transcription". If the voice service is down, Settings → *Voice input* says so; mention it and move on.
- **Google says the account needs reconnecting** (testing-mode tokens expire after 7 days): don't reconnect live. Reconnect the morning of the talk. As a live fallback, show the event that was booked during the rehearsal on the Calendar page.
- **Meeting creation fails:** stop at the summary Marshal shows before booking (don't reply yes). That summary is the architectural point: two-phase, nothing happens unconfirmed.

Transition

"That's what a user sees. Ahtashamul will close by showing what happens underneath, and generating a finished document from the project's sources."

5

Under the hood, a generated document, and what we learned

Ahtashamul Haque (2105067) · Backend architecture and generation lead · 16:00 – 20:00

Time budget: Start generation, explain the backend 1:30 · Roles and security while it runs 0:45 · Open the generated PDF 0:30 · Results, limits, takeaways, close 1:15

Script length: about 369 spoken words, roughly 2.7 minutes of speech; the rest is demo time.

What to show

- **Generate Doc** on Padma View Specialised Hospital Extension → **Tender Summary** → **Generate PDF**, started first and running while he talks.
- **Company Settings** → **User Roles**: the built-in roles and a custom role's permissions (while it generates).
- The finished PDF with its citations, then the closing points.

Architecture elements to explain

- **FastAPI application:** a canonical notebook, exported to `run_backend.py` for headless runs. Every feature module has a `register_*_routes(namespace)` entry point.
- **Account isolation:** `require_account` → `AccountContext` is a dependency of every route, so each account has its own rows, files and vector index.
- **PostgreSQL:** one table per entity, typed columns plus `jsonb`. Saves write only the rows that changed, multi-record changes are atomic, and schema migrations are versioned.
- **Generator abstraction:** one `v1_generate` function behind `CLIProxyAPI` (OpenAI-compatible), so the model can be swapped without touching features.
- **Document generation:** per-section retrieval → structured composition → citation validation → ReportLab PDF.

What to say

Thank you, Shafi. I'm Ahtashamul, and I worked on the backend and document generation. [\[Open Padma View Specialised Hospital Extension → Generate Doc → Tender Summary → Generate PDF.\]](#) I'm starting a Tender Summary now, because it takes about a minute and a half, so let me use that time to show you what's underneath.

The backend is one FastAPI application. It started life as a Kaggle notebook, and we kept the notebook as the source of truth and export it to a plain Python server. Everything that doesn't need a GPU lives in its own module and plugs in through one registration function. That's what let us write over seven hundred tests that run without a GPU at all.

Every route begins the same way. The session token is resolved into an account context, and the handler only ever sees that account's workspace. All records live in PostgreSQL, one table per entity, and changes that touch several records, like committing an onboarding draft, happen in a single transaction, so a failure leaves nothing half-done. Passwords are hashed with PBKDF2 at six hundred thousand iterations, and OAuth tokens are encrypted.

[\[Close the dialog with ✕; it keeps generating. Open Company Settings → User Roles.\]](#)

Permissions are data, not code. There are two built-in roles, Head (Super Admin) and Head (System Admin). Every other role is exactly the permissions ticked here, and the server checks them on every

request. Managing roles is deliberately not a permission, so it can never be handed out by accident.

[The PDF has downloaded; open it from the browser's download bar.] Here's the Tender Summary. Each section was written from pages retrieved for that section. Every citation is checked against the sources the model was actually given, so it can't cite something it never saw. Where the documents are silent, it says so.

To close. We set out to answer from a project's own documents with evidence, and to manage the project in the same place, safely, for several companies at once. Nine of our ten objectives are met and verified. Microsoft 365 is built, but not yet tested against a live account. What we'd do next: move the vectors into PostgreSQL with pgvector, make retrieval adapt to how much text a page has, and test on real scanned drawings. The lesson we take away is to treat every claim, including our own, as something to measure. Thank you. We're happy to take your questions.

Demo steps

1. Open **Padma View Specialised Hospital Extension** → header **Generate Doc** → Document Type **Tender Summary** → **Generate PDF** (the button changes to *Generating...*). Start this in your first 15 seconds. It took 80–85 s in rehearsal.
2. While it runs: close the dialog with × (generation carries on, and the PDF still downloads when ready; verified in rehearsal) → **Company Settings** → **User Roles**. Point at the two built-in roles, then open one custom role to show its permission checkboxes. Don't change anything.
3. When it finishes, the dialog closes and the PDF downloads by itself → open it from the browser's download bar → scroll to one cited section.
4. Final 75 seconds: no navigation. Face the audience for the closing points.

If it fails

- **Generation takes more than 2 minutes or fails** (model proxy): open the Tender Summary generated during the rehearsal, kept in the Downloads folder. Say "here's one we generated this morning".
- **PDF viewer blocked by a download manager (IDM)**: the app shows a notice; open the downloaded file from the Downloads folder instead.
- **Running over time**: drop the User Roles visit (Enayet already covered permissions) and go straight to the PDF and the close.

Transition

Close: "Thank you. We're happy to take your questions." Everyone steps forward for Q&A.

Demo cheat sheet — exactly what to type

Question to ask	Expected answer (from the pack)	Used by
What is the RFI response period on Padma View?	7 working days; 3 working days if it affects the critical path (marked Urgent)	Mukdho
What are the working hours on site for Padma View?	08:00 to 18:00, Saturday to Thursday; Friday is the weekly holiday	Shafi (voice)
What concrete grade is used for the piles on Padma View?	4000 psi (C30/37), 75 mm cover, sulfate-resisting cement	Spare, if time allows

Item	Value
Live project	Bashundhara Clinic Annex · code BCA-LIVE · today → +3 months
Live task	Site mobilisation · first try one day before the project start (refused), then inside
File to upload	demo/project-pack/09_Site-Instruction_SI-014.docx
Document to link	06_Tender-Specification.pdf (already on the Documents page)
Meeting sentence	Schedule a Google Meet called RFI log review with tomorrow at 10am
Document template	Tender Summary, on Padma View Specialised Hospital Extension
Health check	http://127.0.0.1:8000/api/health → healthy, database connected, voice available
Backup server	<code>python demo/demo_server.py --port 8901</code> → http://127.0.0.1:8901

Numbers worth remembering

- Retrieval experiment: Hit@5 **0.95**, **19/20** answers correct, median answer time $\approx$ **6.9 s** (20 questions, 13-page fictional pack).
- **31** permissions; two built-in roles, Head (Super Admin) and Head (System Admin).
- **703** automated backend tests on real PostgreSQL, plus browser suites (UI sweep 58/58, permissions 33/33, API journey 36/36, UI journey 11/11).
- **9 of 10** objectives met and verified; Microsoft 365 built but not yet tested against a live account.
- Voice recordings: up to **60 s** / 20 MB, transcribed locally by Whisper *base*.

Golden rules for the live demo

- The person speaking drives the laptop. Hand over the mouse at each transition, not mid-sentence.
- Narrate while you wait. Never stand silent while a page loads or a model answers.
- If something fails, say one honest sentence ("the model proxy is slow today"), use the backup in your section, and keep going. Don't debug live.
- Stay on time. Each section has a minute budget. The next speaker gives a small hand signal at 3:30.

Questions & answers — who takes what

Answer in two or three sentences, then stop. If a question belongs to someone else, hand it over by name.

Likely question	Who answers	Key points
Why read pages as images with ColPali, instead of just extracting text?	Mukdho	Drawings, title blocks and scanned tables have little extractable text; ColPali embeds the page image. The lexical channel still catches exact clause wording.
How do you stop the model from making things up?	Ahtashamul / Mukdho	It answers only from supplied pages, with numbered sources; citations are validated against those sources; it says when the evidence is silent; the evidence viewer shows the page.
How accurate is it?	Mukdho	Hit@5 of 0.95 and 19/20 correct answers on a 20-question, 13-page fictional pack; median answer time about 7 s. A small set, enough to test design choices, not to rank retrievers.
How is one company's data kept from another's?	Ahtashamul	Every route depends on <code>require_account</code> ; records are keyed by account in PostgreSQL; files and the vector index sit in per-account folders; 33 permission and isolation checks in the browser suite.
Why PostgreSQL, when it began with JSON files?	Ahtashamul	Database-enforced uniqueness (e.g. emails), transactions across stores, concurrent processes; the real data was migrated and verified record for record.
What does it cost to run? Is it tied to one model?	Ahtashamul	Retrieval runs on a local 8 GB GPU; generation goes through one <code>v1_generate</code> function behind an OpenAI-compatible proxy, so the provider can be swapped. Development used a proxy not suited to commercial use.
What about Microsoft 365?	Shafi	Implemented (OAuth, OneDrive, Outlook mail and calendar, Teams) and unit-tested, but not yet tested against a live Microsoft account. It's listed as a limitation.
How were roles and permissions designed?	Enayet	31 permissions, down to task fields and cost types; two built-in roles; every other role is exactly its ticked permissions; managing roles is not itself a permission.
How do you know it works?	Tonmoy	703 automated backend tests on real PostgreSQL, plus browser suites driving the real UI (UI sweep 58/58, API journey 36/36, UI journey 11/11, permissions 33/33, voice, onboarding, documents and dates).